

提示詞 150 模板合輯

《Claude Code Pro》Appendix A — 14 大類 (A-N) 共 150 個範本
列印用速查表 | 每個模板含 [佔位符] · 填入後即可送出

A-核心框架 10 個

A-01 五要素提示詞結構

角色：擔任資深 [語言/框架] 開發者。
情境：這是一個使用 [技術堆疊] 的 [框架] 專案。相關檔案為 [檔名]。
任務：[具體的命令式任務描述]。
限制：[隱含需求或模式，如「遵循現有命名慣例」]。
成功標準：[預期輸出格式或驗收條件]。

A-02 情境設置標準格式

在開始之前，請先理解以下情境：

- 專案類型：[Web App / CLI / API / 函式庫]
- 技術堆疊：[主要語言、框架、資料庫]
- 目前問題：[描述現狀]
- 目標：[描述期望結果]
- 限制條件：[不能改動的部分、必須相容的版本]

請先複述你的理解，確認後再開始動手。

A-03 多步驟任務分解

請將以下任務分解為獨立步驟，每步驟不超過 30 行程式碼：

任務：[任務描述]

要求：

1. 列出完整步驟清單（含依賴順序）
2. 標示哪些步驟可以平行執行
3. 每步驟完成後再繼續下一步

不要一次寫完所有程式碼。

A-04 漸進式開發請求

請用鷹架式開發法實作 [功能]：

第一階段：只寫函式簽名 + docstring（不寫實作）
→ 等我確認架構後再繼續

第二階段：逐一填入每個函式的實作
→ 每填完一個就停下來等確認

第三階段：加入錯誤處理與邊界案例

A-05 角色扮演框架

請扮演一位有 10 年 [領域] 經驗的資深工程師。
你剛加入我們的團隊，正在審查這段程式碼：
[貼上程式碼]

請用你的經驗告訴我：

1. 這段程式碼最嚴重的問題是什麼？
2. 你會如何重構它？
3. 有哪些隱藏的風險我可能沒發現？

A-06 技術決策諮詢

我需要在以下方案中做選擇，請幫我分析：

方案 A：[描述]

方案 B：[描述]

方案 C（如果有）：[描述]

評估標準（依重要性排序）：

1. [最重要的標準，如效能]
2. [次重要，如維護性]
3. [第三，如學習曲線]

請給出明確建議，不要模糊回答「都可以」。

A-07 反向驗證法

我認為 [你的結論/設計決策] 是正確的。
請幫我找出這個結論的漏洞：

- 有哪些邊界條件會讓它失敗？
- 有哪些假設我沒有明說但其實依賴了？
- 有哪些更好的替代方案我可能沒考慮到？

請直接批評，不要客氣。

A-08 輸出格式規範

實作 [功能] 並按以下格式輸出：

1. 先列出【將會建立/修改的檔案清單】
2. 等我確認後，逐檔輸出程式碼（每次一個檔案）
3. 每個檔案結束後標註「[檔名] 完成，繼續？」
4. 最後列出需要執行的指令（安裝、遷移、測試）

A-09 程式碼理解請求

請解釋這段程式碼的運作方式：
[貼上程式碼]

解釋層次：

1. 高層次：這段程式碼的目的是什麼？（1-2 句）
2. 流程：執行順序是什麼？（用流程圖或編號步驟）
3. 關鍵決策：有哪些設計選擇值得注意？
4. 潛在問題：有什麼地方可能出錯？

A-10 首次任務協議

這是我們第一次合作。在開始任何任務之前：

1. 讀取專案的 CLAUDE.md（如果存在）
2. 掃描 README.md 了解專案結構
3. 列出你觀察到的技術堆疊與慣例
4. 確認你已理解，再等我下達第一個實際任務

B-功能開發 15 個

B-01 新功能鷹架

建立 [功能名稱] 的完整鷹架。

按以下順序建立檔案：資料庫 Schema → 查詢函式 → 服務層 → API 端點 → 前端元件 → 測試。

在寫程式碼之前先列出檔案清單。

遵循 CLAUDE.md 與現有程式碼慣例。

B-02 CRUD 操作生成

為 [資源名稱] 資源產生完整 CRUD：

- 資料模型：[欄位清單與型別]
- 框架：[Express/Next.js/FastAPI/Django 等]
- 資料庫：[PostgreSQL/MySQL/MongoDB 等]

包含：建立、讀取（單筆+列表+搜尋）、更新、刪除。

每個端點要有輸入驗證與適當的 HTTP 狀態碼。

B-03 表單驗證實作

為 [表單名稱] 實作前後端共用的表單驗證：

欄位：[欄位名稱、型別、驗證規則]

要求：

- 使用 `Zod` (或 [你指定的函式庫]) 定義 `schema`
- 前端：即時錯誤提示 (`blur` 事件觸發)
- 後端：API 端點也要驗證 (不信任前端)
- 錯誤訊息用繁體中文

B-04 認證流程

實作 [JWT / Session / OAuth] 認證系統：

- 註冊：`email + password (bcrypt hash)`
- 登入：驗證憑證 → 發放 `token`
- 保護路由：中介層驗證 `token`
- 登出：清除 `session` / 列入黑名單
- 密碼重置：`email` 連結流程

請確保遵循 OWASP 認證安全最佳實踐。

B-05 檔案上傳功能

實作檔案上傳功能：

- 允許的檔案類型：[`pdf/jpg/png` 等]
- 最大檔案大小：[數字] MB
- 儲存位置：[本地 / S3 / Google Drive]

包含：前端拖放介面、上傳進度條、伺服器端驗證、錯誤處理。

不要在 URL 中暴露原始檔名 (使用 UUID 重命名)。

B-06 搜尋功能

為 [資源] 實作搜尋功能：

- 搜尋欄位：[欄位清單]
- 前端：輸入防抖 (300ms)
- 後端：[全文搜尋 / LIKE / Elasticsearch]
- 支援：[模糊搜尋 / 精確匹配 / 篩選器]
- 效能：每次請求不超過 200ms (加索引)

包含高亮顯示搜尋關鍵字的功能。

B-07 分頁功能

為 [端點或列表] 實作分頁：

方式：[Cursor-based (推薦大資料集)] / Offset-based]

預設每頁：[數量] 筆

API 回傳格式：

```
{
  data: [...],
  pagination: { total, page, limit, hasNext,
nextCursor }
}
```

前端：無限捲動 或 頁碼導航 (擇一)。

B-08 通知系統

實作 [Email / 推播 / 站內信] 通知系統：

觸發事件：[事件清單，如「訂單完成」「系統警告」]

通知範本：[各事件的訊息格式]

要求：

- 非同步發送 (不阻塞主要流程)
- 失敗重試 (最多 3 次，指數退避)
- 記錄發送日誌 (成功/失敗)

B-09 角色與權限系統

實作 RBAC (角色為基礎的存取控制)：

角色清單：[`admin / editor / viewer` 等]

資源清單：[各角色能操作的功能]

實作方式：

- 資料庫：`roles, permissions, user_roles` 資料表
- 中介層：`checkPermission(resource, action)`
- 前端：根據權限隱藏/停用 UI 元素

B-10 API 客戶端生成

為 [外部 API 名稱] 建立 API 客戶端封裝：

基礎 URL：[URL]

認證方式：[API Key / OAuth / Bearer Token]

要包含的端點：[端點清單]

要求：

- 統一的錯誤處理
- 自動重試 (429 Too Many Requests)
- 請求/回應日誌 (開發模式)
- TypeScript 型別定義

B-11 WebSocket 即時功能

實作 WebSocket 即時功能：

功能：[聊天室 / 即時通知 / 協作編輯 等]

框架：[Socket.io / 原生 WebSocket / Server-Sent

Events]

要包含：

- 連線/斷線處理
- 自動重連 (指數退避)
- 房間管理 (如適用)
- 事件型別的 TypeScript 定義

B-12 排程任務

建立定期排程任務：

任務名稱：[任務描述]

執行頻率：[Cron 表達式，如 "0 9 * * 1-5"]

執行內容：[步驟清單]

要求：

- 避免重複執行 (分散式鎖)
- 執行日誌 (開始/結束/耗時)
- 失敗告警 (寄 email 或 Telegram)
- 可手動觸發 (用於測試)

B-13 匯入/匯出功能

實作 [CSV / Excel / JSON] 資料匯入/匯出：

匯出：

- 觸發：前端按鈕點擊
- 包含欄位：[欄位清單]
- 檔名格式：[資源名]_[日期].csv

匯入：

- 驗證每一列的格式與必填欄位
- 顯示匯入結果 (成功 N 筆 · 失敗 M 筆 + 原因)
- 失敗列可下載為錯誤報告

B-14 多語系支援 (i18n)

為現有專案加入多語系支援：

目標語言：[語言清單，如 zh-TW, en, ja]

框架：[next-intl / react-i18next / 原生]

要求：

- 提取現有硬編碼字串到語系檔
- URL 語系前綴 (/zh-TW/... / /en/...)
- 語言切換元件
- 日期、數字、貨幣格式化

B-15 軟刪除功能

為 [資源] 實作軟刪除 (Soft Delete)：

- 新增 `deleted_at` 欄位 (TIMESTAMP，可為 null)
- 所有查詢預設排除已刪除記錄
- 提供 `restore` (還原) 端點
- 管理介面可查看並還原已刪除記錄
- 可選：設定自動永久刪除 (30 天後)

C-04 資料庫查詢除錯

這個 SQL 查詢執行緩慢或返回錯誤結果：

[貼上 SQL]

表格結構：[貼上 DESCRIBE 或 schema]

資料量：[筆數]

請分析：

1. 執行計畫 (EXPLAIN 輸出解讀)
2. 效能問題 (是否缺少索引?)
3. 邏輯問題 (JOIN 條件是否正確?)
4. 優化後的 SQL

C-05 前端渲染問題

這個 React 元件有渲染問題：[描述問題，如「無限重渲染」、「畫面閃爍」、「狀態不更新」]

[貼上元件程式碼]

請診斷：

1. `useEffect` 的依賴陣列是否正確?
2. 哪些狀態變更觸發了不必要的重渲染?
3. 是否有 `stale closure` 問題?
4. 修復後的程式碼

C-除錯與診斷 12 個

C-01 標準除錯協議

除錯這個錯誤：

錯誤訊息：[貼上完整錯誤 + stack trace]

重現步驟：[步驟清單]

環境：[OS / Node 版本 / 瀏覽器 等]

請按以下步驟回應：

步驟 1：根本原因分析

步驟 2：修復方案

步驟 3：這個錯誤本來應該被哪個測試抓到?

步驟 4：如何防止類似錯誤再次發生?

C-02 效能瓶頸診斷

這個 [函式/API/頁面] 執行太慢 (目前：[時間]ms，目標：[時間]ms)。

請協助診斷：

1. 識別最可能的瓶頸 (N+1 查詢? 重複計算? 同步阻塞?)
 2. 建議 `profiling` 方法 (如何量測每個步驟)
 3. 提出優化方案 (從最高 ROI 開始)
 4. 預估每個方案的改善幅度
- [貼上程式碼或 SQL]

C-03 非同步錯誤追蹤

這個非同步函式有時會靜默失敗，找不到錯誤日誌：

[貼上程式碼]

請幫我：

1. 找出所有可能吞掉錯誤的地方 (空的 `catch`、未處理的 `Promise rejection`)
2. 加入適當的錯誤日誌 (含錯誤類型、相關參數、時間戳)
3. 加入錯誤上報 (Sentry 或 `console.error`)
4. 確保所有錯誤都會往上傳遞

C-06 環境差異診斷

這段程式碼在本機正常，但在 [staging/production/CI] 環境失敗：

錯誤：[貼上錯誤訊息]

本機環境：[OS, 版本]

目標環境：[OS, 版本]

請幫我找出可能的差異：

- 環境變數差異
- 路徑分隔符差異 (Windows vs Linux)
- 時區差異
- 相依套件版本差異

C-07 API 回應錯誤診斷

呼叫 [API 端點] 時收到錯誤：

HTTP 狀態碼：[狀態碼]

回應內容：[貼上回應]

請求程式碼：[貼上程式碼]

請分析：

1. 是請求格式錯誤還是伺服器問題?
2. 請求標頭是否正確 (Content-Type, Authorization)?
3. 請求本體格式是否符合 API 文件?
4. 正確的請求寫法

C-08 記憶體洩漏偵測

這個 [Node.js / 瀏覽器] 應用程式記憶體持續增長：

症狀：[描述，如「運行 2 小時後記憶體超過 1GB」]

[貼上懷疑的程式碼]

請協助：

1. 識別常見洩漏模式 (事件監聽器未移除、全局變數、閉包)
2. 建議用 Chrome DevTools / clinic.js 定位的步驟
3. 修復建議

C-09 競態條件偵測

這段非同步程式碼偶爾會出現不一致的資料（競態條件）：
[貼上程式碼]
場景：[描述觸發條件，如「多個使用者同時提交」]
請找出：

1. 競態條件發生的具體時序
2. 修復方式（鎖、佇列、樂觀鎖、資料庫 transaction）
3. 加入測試模擬並發場景

C-10 第三方套件衝突

安裝 [套件名稱@版本] 後出現衝突：
錯誤訊息：[貼上]
目前 package.json 相關依賴：[貼上]
請分析：

1. 哪個套件造成版本衝突？
2. 安全的解決方案（升級/降級/resolutions）
3. 如果無法解決，有哪些替代套件？

C-11 日誌分析

從這份日誌中找出問題：
[貼上日誌內容]
時間範圍：[開始時間] 到 [結束時間]
現象：[描述問題，如「API 延遲突然升高」]
請提取：

1. 異常模式（頻率、時間點）
2. 相關的錯誤訊息
3. 可能的根本原因
4. 建議的後續調查步驟

C-12 部署錯誤診斷

部署後發現問題：
部署方式：[Docker / Vercel / GitHub Actions 等]
錯誤症狀：[描述]
部署日誌：[貼上]
請協助：

1. 從日誌找出錯誤的根本原因
2. 是應用程式問題還是環境設定問題？
3. 緊急回滾的步驟
4. 根本修復方案

D-02 單元測試生成

為以下函式撰寫單元測試：
[貼上函式]
測試案例需包含：
1. 正常案例（happy path）
2. 邊界值（空值、null、最大/最小值）
3. 錯誤案例（預期拋出的例外）
4. 外部依賴請使用 mock（不呼叫真實 API）

D-03 整合測試

為 [模組名稱] 撰寫整合測試：
測試對象：[描述要測的整合點，如「使用者 API + 資料庫」]
測試環境：使用測試資料庫（不影響正式環境）
包含：
- 測試前置作業（seed 測試資料）
- 測試後清理（rollback 或 truncate）
- 覆蓋主要使用流程（不只是 happy path）

D-04 E2E 測試

用 Playwright 為以下流程撰寫 E2E 測試：
流程：[描述，如「使用者註冊 → 登入 → 建立文章 → 登出」]
要求：
- 使用 Page Object Model 模式
- 每個步驟加入適當的 assertion
- 截圖保存失敗的狀態
- 可在 CI 環境執行（headless 模式）

D-05 API 端點測試

為以下 API 端點撰寫測試：
端點：[HTTP 方法 + 路徑]
要測試的案例：
- 正常請求（各種有效輸入）
- 認證失敗（401）
- 權限不足（403）
- 輸入驗證失敗（400，列出各欄位錯誤）
- 資源不存在（404）
- 伺服器錯誤（模擬資料庫斷線）

D-06 邊界值測試

為 [函式/模組] 生成邊界值測試案例：
輸入型別：[字串/數字/陣列/物件]
請涵蓋：
- 空字串、空陣列、空物件
- null 與 undefined
- 負數、零、最大安全整數
- 超長字串（10000 字元）
- 特殊字元（<script>、SQL 注入字串、Unicode）
- 型別錯誤（傳入錯誤型別）

D-測試策略 12 個

D-01 完整測試套件生成

為 [檔名] 撰寫完整的測試套件：
涵蓋：正常路徑、所有錯誤案例、null/空值邊界條件。
測試行為而非實作，目標覆蓋率 95%+。
使用 [Jest / Pytest / Vitest 等]，每個測試案例要有描述性的名稱。

D-07 效能測試

為 [API/函式] 撰寫效能測試：

工具：[k6 / Artillery / autocannon]

測試情境：

- 基準測試：1 個使用者，持續 30 秒
- 負載測試：100 個並發使用者，持續 5 分鐘
- 壓力測試：逐步增加到 500 使用者

驗收標準：

- P95 延遲 < [X] ms
- 錯誤率 < [Y]%
- 每秒處理 > [Z] 個請求

D-08 測試資料生成

為 [資源名稱] 的測試撰寫資料工廠 (Factory)：

欄位：[欄位清單與型別]

要求：

- 使用 [Faker.js / factory_boy / 等] 生成隨機但合理的測試資料
- 提供常用的固定情境 (如「已驗證使用者」、「已過期訂閱」)
- 可以客製化特定欄位，其餘隨機生成

D-09 Mock 與 Stub 設計

為以下外部依賴建立 Mock：

依賴：[第三方 API / 資料庫 / 郵件服務等]

要求：

- 模擬成功回應 (各種有效情境)
- 模擬失敗回應 (網路錯誤、API 錯誤、超時)
- Mock 要能在測試間重置 (避免狀態污染)
- 使用 [Jest Mock / unittest.mock /nock 等]

D-10 測試覆蓋率提升

目前 [模組] 的測試覆蓋率只有 [X]%，目標是 [Y]%。

[貼上覆蓋率報告或未覆蓋的程式碼]

請：

1. 找出未覆蓋的關鍵路徑
2. 優先補足高風險的錯誤處理分支
3. 撰寫缺少的測試案例

D-11 回歸測試

[功能/模組] 最近發生了一個 bug：[描述 bug]

修復後，請為這個 bug 撰寫回歸測試：

- 測試名稱要清楚描述是哪個 bug 的回歸測試
- 在修復前，這個測試應該會失敗
- 在修復後，這個測試應該會通過
- 加入注釋說明 bug 的根本原因

D-12 測試重構

這些測試太難維護了：[描述問題，如「重複設定程式碼」、「測試之間相依」、「測試名稱不清楚」]

[貼上現有測試]

請重構：

- 提取共用的 setup/teardown
- 使用參數化測試 (describe.each / @pytest.mark.parametrize)
- 改善測試名稱 (描述行為，不描述實作)
- 確保測試順序無關 (可以獨立執行)

E-重構與品質 10 個

E-01 Audit → Protect → Transform

Phase 1 → 審計：分析 [函式名稱]。

列出：職責、依賴關係、假設前提、狀態變更。

標記已知的 bug (// 特徵：...) 並標記每個步驟。

Phase 2 → 保護：撰寫特徵測試。

涵蓋每個 // 特徵標記。

Phase 3 → 轉換：逐步重構。標記每個步驟。

// 步驟：每步驟執行測試。

E-02 函式拆解

這個函式太長了 ([N] 行)，做了太多事情：

[貼上函式]

請拆解為：

- 每個子函式只做一件事 (不超過 20 行)
- 函式名稱要能自我說明 (不需要注釋)
- 提取前先行列出拆解計畫，確認後再動手

E-03 移除重複碼

這些程式碼有重複邏輯：

[貼上程式碼]

請：

1. 識別重複的模式
2. 提取為共用函式/元件/模組 (放在哪裡?)
3. 確保提取後的行為與原本完全一致
4. 更新所有呼叫點

E-04 型別安全升級

將以下 JavaScript / 弱型別 Python 程式碼升級為強型別：

[貼上程式碼]

要求：

- TypeScript：加入完整的 interface / type / generic
- Python：加入 type hints + Pydantic model
- 消除所有 any / Any / Dict[str, Any]
- 確保 TypeScript strict mode 通過

E-05 錯誤處理強化

這段程式碼的錯誤處理不足：

[貼上程式碼]

請加入：

- 所有可能拋出例外的地方 (I/O、API 呼叫、型別轉換)
- 具體的錯誤型別 (不用裸 except)
- 有意義的錯誤訊息 (含相關的輸入值)
- 區分「可恢復錯誤」與「不可恢復錯誤」的處理方式

E-06 命名改善

這段程式碼的命名太難理解：

[貼上程式碼]

請改善所有命名：

- 變數名稱描述內容 (d → roomPm25Data)
- 函式名稱描述動作 (get → fetchRoomSensorData)
- 布林值用 is/has/can 開頭 (flag → isAuthenticated)
- 避免縮寫 (tmp, res, cb, cb2)

E-07 非同步轉換

將以下 `callback` 風格或 `Promise` 鏈的程式碼轉換為 `async/await`：
[貼上程式碼]
要求：

- 保留所有錯誤處理語意
- 並行的操作繼續使用 `Promise.all`
- 加入適當的 `try/catch`
- 轉換後功能完全一致（加測試驗證）

E-08 效能重構

這段程式碼有效能問題：
[貼上程式碼]
問題描述：[如「`N+1` 查詢」、「 $O(n^2)$ 複雜度」、「重複計算」]
請重構並解釋：

1. 問題在哪裡（為什麼慢）
2. 修復方案（改用什麼演算法/模式）
3. 預期改善幅度
4. 如何驗證改善效果

E-09 模組化重構

這個單一大檔案（`[N]` 行）需要拆分：
[描述檔案功能]
請規劃拆分方案：

- 建議的模組邊界（每個模組的職責）
- 模組間的依賴關係（避免循環依賴）
- 建議的目錄結構
- 移植順序（最安全的拆分步驟）

E-10 可讀性優化

這段程式碼運作正確但很難讀懂：
[貼上程式碼]
請在不改變功能的前提下提升可讀性：

- 用提前 `return` 減少巢狀（`guard clause`）
- 用常數替代 `magic number`（`42` → `MAX_RETRY`）
- 把複雜條件提取為具名布林變數
- 加入必要的注釋（解釋「為什麼」，不解釋「做什麼」）

F-PR 審查 10 個

F-01 標準 PR 審查

審查這個 `branch` 對 `main` 的 `git diff`。
檢查：

- (1) 正確性：邊界條件、邏輯錯誤
- (2) 安全性：輸入驗證、認證、敏感資料
- (3) `CLAUDE.md` 慣例：命名、結構、風格
- (4) 效能：`N+1` 查詢、不必要的重複計算

給出明確的改進建議，不要只說「看起來不錯」。

F-02 安全性審查

對這個 PR 進行安全性審查，重點檢查：

- SQL 注入（所有資料庫查詢是否參數化？）
- XSS（使用者輸入是否有跳脫？`innerHTML` 的使用？）
- 敏感資料（API Key、密碼是否被記錄到日誌？）
- 認證繞過（所有端點是否有正確的驗證？）
- 不安全的直接物件引用（`IDOR`）

列出每個問題的嚴重程度（嚴重/高/中/低）。

F-03 效能審查

審查這個 PR 的效能影響：

- 是否有新增 `N+1` 查詢？
- 是否有在迴圈中執行資料庫查詢？
- 是否有不必要的資料載入（應該使用分頁？）
- 前端：是否有不必要的重渲染？
- 是否有大型資料在記憶體中操作？

估計最壞情境下的效能影響。

F-04 測試覆蓋審查

審查這個 PR 的測試品質：

- 新增的功能是否有對應的測試？
- 測試是否覆蓋了錯誤路徑？
- 測試是否可以獨立執行（無狀態依賴）？
- 有哪些邊界條件沒有被測試？

列出「應該要有但沒有」的測試案例。

F-05 破壞性變更檢查

這個 PR 是否有破壞性變更？
請檢查：

- API 回傳格式是否有變更（刪除欄位、改名、改型別）？
- 資料庫 `schema` 是否需要遷移？
- 現有的 API 客戶端是否需要更新？
- 環境變數是否有新增或改名？

如果有破壞性變更，列出影響範圍和遷移步驟。

F-06 架構合規審查

審查這個 PR 是否符合 `CLAUDE.md` 定義的架構：

- 新增的程式碼放在正確的層級（`controller/service/repository`）？
- 是否違反了既有的分層邊界？
- 命名是否遵循專案慣例？
- 是否有不應該直接存取資料庫的地方（應透過 `service` 層）？

F-07 依賴審查

這個 PR 新增了以下依賴：[清單]
請評估：

- 這些套件的維護狀態（最後更新時間？Issue 數量？）
- 授權是否與專案相容？
- 套件大小對 `bundle size` 的影響？
- 有沒有更輕量的替代方案？
- 有已知的安全漏洞嗎？（`npm audit`）

F-08 文件完整性審查

審查這個 PR 的文件是否完整：

- 新增的 API 端點是否有 JSDoc / docstring？
- README 是否需要更新？
- CHANGELOG 是否有記錄？
- 複雜的邏輯是否有解釋「為什麼」的注釋？
- 新的環境變數是否已加入 `.env.example`？

F-09 回滾計畫審查

如果這個 PR 上線後出問題，我們能如何回滾？

請評估：

- 是否有資料庫 schema 變更（有 down migration 嗎？）
- 回滾後是否有資料不一致的風險？
- Feature flag 是否可以快速關閉？
- 回滾步驟是什麼？預估需要多久？

F-10 小 PR 拆分建議

這個 PR 太大了（[N] 個檔案，[M] 行變更），難以審查。

請建議如何拆分為較小的 PR：

- 每個小 PR 的功能範圍
- 合併順序（依賴關係）
- 哪些部分可以獨立合併並自行測試

保留原本的功能目標，只是拆分實作步驟。

G-資料庫 12 個

G-01 Schema 設計

為以下需求設計資料庫 Schema：

需求（白話文）：[描述]

資料庫：[PostgreSQL / MySQL / MongoDB]

請設計：

- 資料表清單與欄位（含型別、限制、預設值）
- 外鍵關係
- 索引策略（哪些欄位需要索引）
- 考慮以下查詢模式：[查詢清單]

G-02 遷移腳本生成

根據以下 Schema 變更，生成資料庫遷移腳本：

變更描述：[如「在 users 表新增 last_login_at 欄位」]

要求：

- Up migration（套用變更）
- Down migration（回滾變更）
- 大型資料表（> 100 萬筆）需考慮鎖定問題
- 遷移後加入驗證步驟（確認資料正確）

G-03 查詢優化

優化以下 SQL 查詢（目前執行時間：[X] 秒）：

[貼上 SQL]

[貼上 EXPLAIN ANALYZE 輸出]

資料量：[筆數] 筆

請提供：

1. 問題診斷（為什麼慢？）
2. 優化後的 SQL
3. 是否需要新增索引（哪個欄位？）
4. 預期改善幅度

G-04 索引策略

為 [資料表] 設計索引策略：

常用查詢模式：

1. [查詢 1]
2. [查詢 2]
3. [查詢 3]

請分析：

- 哪些欄位需要索引（單欄位 vs 複合索引）
- 複合索引的欄位順序（選擇率高的放前面）
- 現有索引是否有冗餘可以刪除
- 索引對寫入效能的影響

G-05 種子資料生成

為 [資料表清單] 生成測試種子資料：

資料量：[N] 筆（要夠大到能測試效能）

要求：

- 資料要真實感（不是全 "test1", "test2"）
- 覆蓋各種狀態（active/inactive，各種角色等）
- 相關資料要有正確的外鍵關係
- 使用 [Faker.js / factory_boy / SQL INSERT 等]

G-06 資料備份策略

設計 [PostgreSQL / MySQL] 資料庫備份策略：

資料量：[大小]

RPO（可接受的資料損失）：[時間]

RTO（可接受的恢復時間）：[時間]

請規劃：

- 備份頻率與類型（全量 / 增量）
- 備份存放位置（異地？雲端？）
- 自動化備份腳本
- 定期恢復演練步驟

G-07 資料驗證與清理

[資料表] 的資料有品質問題：

已知問題：[如「email 欄位有無效格式」「外鍵有孤兒資料」]

請生成：

1. 診斷查詢（找出所有有問題的資料）
2. 清理腳本（修復或標記問題資料）
3. 驗證查詢（確認清理後的資料品質）

注意：清理腳本要加 BEGIN TRANSACTION + ROLLBACK（先測試）

G-08 批次操作優化

需要對 [資料表] 的 [N] 筆資料進行批次更新/插入：

目前做法：[描述，如「逐筆 INSERT 在迴圈中」]

請優化：

- 使用 batch INSERT（單次插入多筆）
- 批次大小建議（避免 timeout 與記憶體問題）
- 進度追蹤（每 1000 筆 log 一次）
- 失敗重試邏輯

G-09 資料庫 Transaction 設計

這個操作需要 `transaction` 確保資料一致性：
操作步驟：[列出所有步驟，如「扣款 + 建立訂單 + 發送通知」]
請實作：
- 完整的 `transaction` 包裝
- 每個步驟的錯誤處理與回滾
- 避免長 `transaction` (哪些步驟可以移到 `transaction` 外？)
- 鎖定策略 (樂觀鎖 vs 悲觀鎖)

G-10 連線池設計

設計 [資料庫] 的連線池配置：
應用程式：[框架/語言]
預期並發：[N] 個請求
資料庫伺服器規格：[CPU, RAM]
請建議：
- 連線池大小 (最小/最大連線數)
- 連線超時設定
- 連線健康檢查
- 連線洩漏偵測

G-11 多租戶資料隔離

設計多租戶 (`Multi-tenant`) 資料隔離策略：
租戶數量：[預估]
資料敏感度：[高/中/低]
評估三種方案：
1. 行層級隔離 (每個資料表加 `tenant_id`)
2. `Schema` 隔離 (每個租戶獨立 `Schema`)
3. 資料庫隔離 (每個租戶獨立資料庫)
基於我的需求推薦最適合的方案並說明原因。

G-12 資料庫效能監控

為 [資料庫] 設置效能監控：
要監控的指標：
- 慢查詢 (超過 [X] ms 的查詢)
- 連線池使用率
- 死鎖發生頻率
- 資料表/索引大小增長
請生成：
- 監控查詢 SQL
- 告警閾值建議
- 整合到 [Grafana / Datadog / 自建] 的方法

H-API 串接 10 個

H-01 REST API 客戶端封裝

為 [API 名稱] 建立 REST API 客戶端：
文件 URL：[API 文件連結]
認證：[API Key / Bearer Token / OAuth]
要封裝的端點：[清單]
要求：
- 統一的錯誤處理 (所有 4xx/5xx)
- 自動重試 (429 用 `Retry-After` 標頭)
- 請求/回應日誌 (開發模式)
- `TypeScript` 型別或 `Python type hints`

H-02 OAuth 流程實作

實作 [Google / GitHub / LINE] OAuth 登入：
流程：Authorization Code Flow
要求：
- `state` 參數防 CSRF
- PKCE (如果是 SPA)
- Token 刷新邏輯 (Refresh Token)
- 儲存策略 (`HttpOnly Cookie` · 不存 `localStorage`)
- 連結既有帳號的邏輯 (email 匹配)

H-03 Webhook 接收與驗證

實作 [Stripe / GitHub / LINE] Webhook 接收端點：
Webhook 事件：[事件清單]
要求：
- 驗證 Webhook 簽名 (防偽造)
- 冪等性處理 (重複事件不重複執行)
- 立即回應 200 (非同步處理事件)
- 事件記錄 (含原始 `payload`)
- 失敗告警

H-04 API 錯誤處理策略

為呼叫 [外部 API] 設計完整的錯誤處理策略：
可能的錯誤情境：
- 網路斷線 / 超時
- 429 Rate Limited
- 500 伺服器錯誤 (可重試)
- 401/403 認證失敗 (不可重試)
- 400 請求格式錯誤 (不可重試)
請設計重試策略 (指數退避) 與降級處理。

H-05 GraphQL 查詢建構

為 [GraphQL API] 建構查詢：
需求：[描述要取得的資料]
Schema：[貼上相關的 Type 定義]
請撰寫：
- Query / Mutation / Subscription (依需求)
- Fragment (避免重複欄位)
- 變數定義 (不要 `hardcode` 值)
- `TypeScript` 型別 (用 GraphQL Code Generator)

H-06 API 速率限制處理

呼叫 [API] 時需要遵守速率限制：
限制：[X] 次請求 / [時間單位]
我需要處理 [N] 筆資料，每筆需要 [M] 次請求。
請設計：
- 請求佇列 (控制並發數)
- 速率限制等待 (使用 `Retry-After` 或固定間隔)
- 進度顯示
- 預估完成時間

H-07 API 版本控制

設計 API 版本控制策略：

目前版本：v1 (已有外部使用者)

需要的破壞性變更：[描述]

評估方案：

1. URL 版本 (/api/v2/...)

2. Header 版本 (Accept:

application/vnd.api+json;version=2)

3. 查詢參數 (?version=2)

建議過渡期策略 (v1 維持多久？如何通知使用者？)

H-08 API 文件生成

為現有的 API 端點生成 OpenAPI 3.0 文件：

端點：[貼上程式碼或端點清單]

要包含：

- 每個端點的 summary 和 description
- 請求參數 (path / query / body) 的 schema
- 所有可能的回應 (2xx / 4xx / 5xx)
- 認證方式說明
- 請求/回應範例

H-09 API 快取策略

為以下 API 端點設計快取策略：

端點：[清單]

資料更新頻率：[各端點的資料新鮮度需求]

請設計：

- 快取層 (Redis / 記憶體快取 / CDN)
- 快取鍵設計 (如何避免快取污染)
- TTL 設定 (各端點的過期時間)
- 快取失效策略 (主動刷新 vs 被動過期)

H-10 第三方 API 整合測試

為 [第三方 API] 整合撰寫測試：

要求：

- 使用 Mock 服務 (不呼叫真實 API)
- 測試成功路徑與各種失敗情境
- 測試重試邏輯 (模擬 429 回應)
- 測試 Webhook 接收 (包含簽名驗證)
- CI 環境不需要 API Key 也能執行

I-02 自訂 Hook 生成

建立 [useName] 自訂 Hook：

功能：[描述，如「管理分頁資料獲取」]

參數：[參數清單與型別]

回傳值：[回傳的狀態與函式]

要求：

- 完整的 TypeScript 型別
- 清理副作用 (useEffect cleanup)
- 錯誤處理
- 撰寫對應的單元測試

I-03 資料表格元件

建立具備以下功能的資料表格元件：

欄位：[欄位清單，含標頭、資料鍵、是否可排序]

功能：

- 點擊欄位標頭排序 (升序/降序切換)
- 前端搜尋 (即時過濾)
- 分頁 (每頁 [N] 筆)
- 選取列 (可多選)
- 行動裝置響應式 (水平捲動)

I-04 表單元件

建立 [表單名稱] 表單元件：

欄位：[欄位清單、型別、驗證規則]

使用：React Hook Form + Zod

功能：

- 欄位失焦時驗證 (onBlur)
- 提交時的 loading 狀態
- 成功/失敗的反饋訊息
- 表單重置功能
- 防止重複提交 (disable 按鈕)

I-05 模態視窗元件

建立可重用的模態視窗 (Modal) 元件：

功能：

- 點擊遮罩層關閉
- Escape 鍵關閉
- 焦點捕捉 (Trap Focus，無障礙)
- 動畫效果 (淡入淡出)
- 支援不同尺寸 (sm / md / lg / fullscreen)
- Portal 渲染 (掛載到 body)

I-06 無限捲動

為 [資源] 列表實作無限捲動：

API：[端點 URL 與分頁參數格式]

要求：

- 使用 Intersection Observer (不用 scroll 事件)
- 載入中的 skeleton 畫面
- 已載入全部資料的提示
- 錯誤狀態與重試按鈕
- 捲動位置在頁面刷新後保留 (可選)

I-前端開發 12 個

I-01 React 元件生成

建立 [元件名稱] React 元件：

功能：[描述]

Props：[props 清單與型別]

狀態：[需要管理的狀態]

要求：

- TypeScript (含完整的 prop types)
- Tailwind CSS 樣式 (含響應式)
- 語意化 HTML + ARIA 標籤
- 錯誤狀態與載入狀態
- 匯出型別供外部使用

I-07 拖放排序

為 [清單] 實作拖放排序功能：

函式庫：[dnd-kit / react-beautiful-dnd]

要求：

- 拖動中的視覺回饋（半透明效果）
- 支援鍵盤操作（無障礙）
- 排序後自動儲存到後端
- 樂觀更新（先更新 UI，失敗再回滾）
- 支援觸控設備

I-08 圖表元件

建立 [折線圖 / 長條圖 / 圓餅圖] 元件：

函式庫：[Recharts / Chart.js / D3.js]

資料格式：[描述資料結構]

要包含：

- 響應式（隨容器大小自動調整）
- 懸停 tooltip
- 圖例（Legend）
- 動畫效果
- 無資料時的空狀態
- 資料載入中的 skeleton

I-09 主題切換（深色模式）

為現有 Tailwind CSS 專案加入深色模式：

切換方式：[系統偏好 / 手動切換按鈕]

實作：

- tailwind.config.js 設定（darkMode: 'class'）
- 所有元件加入 dark: 前綴的樣式
- 主題偏好存到 localStorage
- 防止初始載入閃爍（FOUC）
- 切換動畫（300ms 過渡）

I-10 錯誤邊界

為 React 應用加入完整的錯誤處理：

要實作：

- ErrorBoundary 元件（class component）
- 全局未處理 Promise 錯誤捕捉
- 錯誤上報（發送到 Sentry 或自建服務）
- 使用者友善的錯誤畫面（含重試按鈕）
- 開發模式顯示詳細 stack trace

I-11 無障礙設計審查

審查這個元件的無障礙設計：

[貼上元件程式碼]

請檢查並修正：

- 所有互動元素有 ARIA 標籤
- 顏色對比度符合 WCAG AA（4.5:1）
- 鍵盤可以完整操作（Tab 順序正確）
- 螢幕閱讀器可以理解內容
- 表單欄位有 label 關聯

I-12 效能優化

優化這個 React 應用的渲染效能：

問題：[如「列表有 500 筆資料時頁面卡頓」]

請分析並修復：

- 找出不必要的重渲染（用 React DevTools Profiler）
- 實作虛擬化（超過 100 筆資料必須）
- 使用 useMemo/useCallback（只在有意義的地方）
- 程式碼分割（React.lazy + Suspense）
- 圖片懶載入

J-後端架構 10 個

J-01 服務層抽象

為 [功能] 設計服務層（Service Layer）：

要分離的關注點：

- Controller 層：HTTP 請求/回應處理
- Service 層：業務邏輯
- Repository 層：資料存取

請設計各層的介面（interface），確保可以獨立測試。

J-02 中介層設計

設計以下中介層（Middleware）：

需要的中介層：[清單，如「認證、日誌、速率限制、CORS」]

執行順序（重要）：[描述哪些要先執行]

每個中介層要包含：

- 函式簽名
- 錯誤處理
- 如何掛載到 Express / Koa / Fastify

J-03 設定管理系統

為 [應用程式] 設計設定管理系統：

環境：development / staging / production

設定項目：[清單，含敏感度分類]

要求：

- 啟動時驗證所有必要設定（缺少就 crash）
- 敏感值只從環境變數讀取（不寫死）
- 提供 .env.example 範本
- 型別安全（TypeScript 或 Pydantic）

J-04 背景任務佇列

設計背景任務佇列系統：

佇列後端：[Redis / RabbitMQ / AWS SQS]

任務清單：[任務名稱與預估執行時間]

要求：

- 任務重試（失敗後指數退避）
- 任務優先級
- 死信佇列（多次失敗的任務）
- 任務執行狀態查詢
- 監控 Dashboard

J-05 快取策略設計

為 [應用程式] 設計快取策略：

快取層：Redis

要快取的資料：[清單，含更新頻率]

請設計：

- 快取鍵命名規範（避免衝突）
- TTL 設定（各資料類型）
- Cache-Aside vs Write-Through 選擇
- 快取雪崩防護（Stampede Protection）
- 快取失效的觸發點

J-06 日誌系統設計

設計結構化日誌系統：

語言/框架：[Node.js + Winston / Python + structlog 等]

要包含：

- 每個請求的唯一 Trace ID（用於串連日誌）
- 統一的日誌格式（JSON，含時間戳、等級、服務名）
- 敏感資料遮蔽（密碼、Token、個資）
- 日誌等級（DEBUG 只在開發模式）
- 整合到 [Elasticsearch / Loki / CloudWatch]

J-07 健康檢查端點

實作 /health 和 /ready 健康檢查端點：

/health（存活探針）：

- 應用程式是否在運行
- 快速回應（< 10ms）

/ready（就緒探針）：

- 資料庫連線是否正常
- 所有必要的外部服務是否可用
- 可以接受流量了嗎？

格式：{ status, checks: { db, redis, ... }, uptime }

J-08 API 速率限制

為 [API] 實作速率限制：

限制策略：

- 未認證用戶：[X] 次 / 分鐘（按 IP）
- 認證用戶：[Y] 次 / 分鐘（按用戶 ID）
- 特定端點（如登入）：[Z] 次 / 15 分鐘

實作：

- 使用 Redis 儲存計數器
- 回傳 Retry-After header
- 告警（某 IP 持續觸發限制）

J-09 事件驅動架構

將 [功能] 重構為事件驅動架構：

目前做法：[描述同步呼叫的問題，如「訂單完成後同步觸發 5 個動作」]

重構為：事件發布 → 各服務訂閱各自關心的事件

事件清單：[事件名稱與 payload]

實作：[EventEmitter / Redis Pub/Sub / Kafka]

確保：事件消費失敗不影響主要流程。

J-10 微服務拆分規劃

評估是否將 [功能] 從 monolith 拆分為微服務：

目前痛點：[描述，如「部署相互影響」「擴展需求不同」]

請分析：

- 拆分的邊界建議（按業務領域）
- 服務間通訊方式（REST / gRPC / 事件）
- 資料所有權（每個服務管理自己的 DB）
- 拆分的風險與成本
- 是否真的需要微服務？（考慮 modular monolith）

K-DevOps & 部署 10 個

K-01 Dockerfile 最佳化

優化這個 Dockerfile：

[貼上現有 Dockerfile]

請改善：

- 多階段建構（builder + production image）
- 層快取最佳化（依賴先複製）
- 最小化最終映像大小
- 以非 root 用戶運行
- 健康檢查指令
- 建構時不包含敏感資訊

K-02 GitHub Actions CI/CD

建立 GitHub Actions 工作流程：

觸發條件：push to main / PR to main

步驟：

1. 程式碼品質檢查（lint + type check）
 2. 單元測試（含覆蓋率報告）
 3. 整合測試
 4. 建構 Docker 映像
 5. 部署到 [Vercel / GCP / AWS]（只在 main branch）
- 要求：測試通過才部署，機密用 GitHub Secrets。

K-03 環境變數管理

設計跨環境的環境變數管理方案：

環境：local / staging / production

變數清單：[分類：公開/敏感]

方案評估：

1. GitHub Secrets + CI 注入
2. AWS Parameter Store / Secrets Manager
3. Vault
4. .env 檔案（local only）

根據我的規模（[描述]）推薦最適合的方案。

K-04 藍綠部署策略

設計 [應用程式] 的藍綠部署策略：

目前架構：[描述]

目標：零停機部署

請設計：

- 兩套環境的切換機制
- 資料庫遷移的處理（向前相容）
- 流量切換方式（Load Balancer / DNS）
- 回滾步驟（發現問題時如何快速切回）
- 自動化腳本

K-05 監控與告警設置

為 [應用程式] 設置監控與告警：

要監控的指標：

- API 錯誤率 (> 1% 告警)
- P95 延遲 (> 500ms 告警)
- CPU/記憶體使用率 (> 80% 告警)
- 資料庫連線池使用率

工具：[Grafana + Prometheus / Datadog / CloudWatch]

告警通知：[Slack / Email / PagerDuty]

K-06 Docker Compose 開發環境

建立完整的本地開發 Docker Compose 環境：

服務清單：[應用程式、資料庫、Redis、等]

要包含：

- 熱重載 (volume mount 原始碼)
- 資料庫初始化 (自動執行遷移 + seed)
- 環境變數 (.env.local)
- 服務間的網路連線
- 啟動說明 (docker compose up 後需要做什麼)

K-07 自動化部署腳本

建立 [應用程式] 的部署腳本：

部署步驟：

1. 從 Git 拉取最新程式碼
 2. 安裝依賴
 3. 執行資料庫遷移
 4. 建構應用程式
 5. 重啟服務 (zero downtime)
 6. 健康檢查 (確認部署成功)
- 失敗時自動回滾到上一個版本。

K-08 安全掃描整合

在 CI 流程中加入安全掃描：

掃描項目：

- 相依套件漏洞 (npm audit / snyk)
- 原始碼安全問題 (CodeQL / Semgrep)
- Docker 映像漏洞 (Trivy)
- 機密檔案掃描 (git-secrets / detect-secrets)

規則：嚴重 (Critical) 漏洞阻斷部署，高 (High) 警告。

K-09 效能基準測試自動化

在 CI 中自動化執行效能基準測試：

工具：[k6 / Lighthouse CI / autocannon]

測試情境：

- 關鍵 API 端點的延遲基準
- 前端頁面載入速度 (Lighthouse score)

規則：

- P95 延遲退步 > 20% → 阻斷合併
- Lighthouse 分數降低 > 5 分 → 警告

基準值存在 Git 中，PR 時對比。

K-10 Log 聚合設置

設置集中式 Log 聚合系統：

規模：[每天 log 量]

工具：[ELK Stack / Loki + Grafana / Datadog]

要包含：

- Log 收集 Agent 設置
- Log 解析 (structured JSON)
- 保留策略 (熱儲存 7 天，冷儲存 90 天)
- 快速搜尋 (按 TraceID / 用戶 ID)
- 異常 Log 告警

L-安全性 10 個

L-01 全面安全審計

對 [應用程式/功能] 進行安全審計：

請依 OWASP Top 10 逐項檢查：

1. SQL/NoSQL 注入
2. 身份驗證失效
3. 敏感資料暴露
4. XML 外部實體 (XXE)
5. 存取控制失效
6. 安全設定錯誤
7. XSS
8. 不安全的反序列化
9. 使用含漏洞的元件
10. 不足的記錄與監控

每個問題標示嚴重程度並給出修復建議。

L-02 SQL 注入防禦

審查以下程式碼的 SQL 注入風險：

[貼上程式碼]

請找出並修復：

- 所有字串拼接 SQL 的地方
- 改用參數化查詢 (佔位符)
- ORM 的正確使用 (避免 raw query)
- 搜尋功能的特殊字元跳脫

加入測試 (嘗試注入字串驗證已修復)。

L-03 輸入驗證強化

為 [端點/表單] 加入完整的輸入驗證：

輸入欄位：[清單]

驗證規則：

- 型別檢查 (不信任前端的型別)
- 長度限制 (防止過長輸入)
- 格式驗證 (email, URL, 電話)
- 業務邏輯驗證 (值是否在允許範圍內)
- 特殊字元處理 (XSS, SQL 注入)

使用 [Zod / Joi / Pydantic] 實作。

L-04 認證安全強化

強化現有的認證系統：

[描述目前認證方式]

請加入：

- 登入失敗鎖定 (5 次失敗鎖定 15 分鐘)
- 密碼強度規則 (長度、複雜度)
- 安全的密碼重置流程 (有時限的 token)
- 多因素認證 (TOTP)
- 敏感操作要求重新驗證
- Session 在密碼變更後失效

L-05 CORS 設定

為 [API 服務] 設定正確的 CORS：

允許的來源：[清單，如「https://myapp.com」]

要求：

- 不使用 origin: * (除了公開 API)
- 明確設定允許的 HTTP 方法
- 認證請求需設 credentials: true
- 預檢請求 (OPTIONS) 的快取
- 子域名萬用字元的安全性考量

L-06 機密管理

審查程式碼中的機密管理：

[貼上程式碼或目錄結構]

請找出並修復：

- 硬編碼的 API Key / 密碼
- .env 檔案是否被 commit
- 日誌中是否洩漏機密
- 前端 bundle 中是否有敏感設定

建立正確的機密管理流程 (環境變數 + Secret Manager)。

L-07 HTTP 安全標頭

為 [Web 應用程式] 設置安全的 HTTP 標頭：

需要設置：

- Content-Security-Policy (防 XSS)
- X-Frame-Options (防 Clickjacking)
- Strict-Transport-Security (強制 HTTPS)
- X-Content-Type-Options (防 MIME sniffing)
- Referrer-Policy

請生成適合我的應用的 CSP 規則，並說明如何測試。

L-08 依賴漏洞修復

npm audit / pip-audit 發現以下漏洞：

[貼上 audit 報告]

請協助：

1. 評估每個漏洞的實際影響 (是否在我的使用路徑上)
2. 安全的升級路徑 (直接升級/替換套件)
3. 短期緩解措施 (無法立即升級時)
4. 如何在 CI 設置防護 (下次自動偵測)

L-09 IDOR 漏洞防護

審查以下 API 端點的 IDOR (不安全的直接物件引用) 漏洞：

[貼上端點程式碼]

請檢查：

- 是否有驗證當前用戶有權存取該 ID 的資源？
- 是否可以通過修改 ID 存取他人資料？

修復：加入所有權驗證中介層。

測試：撰寫測試案例驗證漏洞已修復。

L-10 滲透測試準備

我們即將進行滲透測試，請幫我準備：

應用程式：[描述]

範圍：[測試範圍]

請生成：

1. 常見的攻擊向量清單 (基於我們的技術堆疊)
2. 需要特別注意的高風險端點
3. 我們應該在測試前自行修復的已知問題
4. 測試環境設置建議 (不要測試正式環境)

M-Agent & MCP workflow 10 個

M-01 MCP 多工具 workflow

使用多個 MCP 工具完成以下任務：

任務：[描述]

可用工具：[GitHub MCP / 資料庫 MCP / 網路搜尋 MCP 等]

步驟：

1. 用 [工具] 取得 [資訊]
2. 用 [工具] 查詢 [資訊]
3. 驗證最新最佳實踐 (網路搜尋 MCP)
4. 呈現實作計畫，等待確認後執行

M-02 Sub-agent 任務分配

這個任務需要多個 Agent 協作完成：

任務：[描述]

請拆分為：

- Agent 1: [職責] (獨立，可平行)
- Agent 2: [職責] (獨立，可平行)
- Agent 3: [職責] (需要等 Agent 1+2 完成)

每個 Agent 的 prompt 要自給自足 (不依賴對話歷史)。

M-03 程式碼探索 workflow

探索 [功能] 在這個 codebase 中的實作：

步驟：

1. 找到入口點 (搜尋關鍵詞: [詞彙])
2. 追蹤執行路徑 (從 controller → service → repository)
3. 找出所有依賴 (這個功能用到哪些模組？)
4. 繪製架構圖 (ASCII 或 Mermaid)
5. 識別可以改進的地方

M-04 自動化測試 Agent

建立一個自動化測試 Agent：

觸發條件：每次 PR 被建立時

Agent 任務：

1. 讀取 PR 的變更內容
2. 識別受影響的功能
3. 撰寫或更新對應的測試
4. 執行測試並回報結果
5. 如果測試失敗，提出修復建議

M-05 程式碼審查 Agent

建立程式碼審查 Agent：

觸發：PR 開啟或更新

審查清單：

- 安全性漏洞
- 效能問題
- CLAUDE.md 慣例符合度
- 測試覆蓋率

輸出格式：

- 必改項目 (阻斷合併)
- 建議改進 (不阻斷)
- 優點表揚 (正向回饋)

M-06 文件生成 Agent

建立自動文件生成 Agent：

輸入：程式碼變更 (git diff)

輸出：

- 更新 API 文件 (OpenAPI)
- 更新 README 中受影響的章節
- 生成 CHANGELOG 條目
- 如有破壞性變更，生成遷移指南

確保文件與程式碼同步更新。

M-07 資料分析 Agent

建立資料分析 Agent：

資料來源：[資料庫 / CSV / API]

分析任務：

1. 載入資料 ([來源描述])
2. 清理資料 (處理缺值、異常值)
3. 計算關鍵指標：[指標清單]
4. 生成視覺化圖表
5. 撰寫分析摘要報告

輸出格式：Markdown 報告 + 圖表檔案

M-08 錯誤監控 Agent

建立錯誤監控與自動診斷 Agent：

觸發：錯誤率超過閾值時

Agent 任務：

1. 取得最近 30 分鐘的錯誤日誌
2. 聚合相似錯誤 (grouping)
3. 找出最常見的根本原因
4. 查詢 codebase 找到相關程式碼
5. 提出修復建議
6. 通知相關開發者 (含診斷報告)

M-09 依賴升級 Agent

建立定期依賴升級 Agent：

觸發：每週一自動執行

任務：

1. 掃描過時的依賴 (npm outdated / pip list --outdated)
2. 評估每個升級的破壞性變更風險
3. 先升級低風險的依賴
4. 執行測試確認無回歸
5. 建立 PR (含升級說明)
6. 高風險升級等待人工審核

M-10 上下文感知助手

在開始任何任務前，先執行以下上下文收集：

1. 讀取 CLAUDE.md (了解專案規則)
2. 讀取最近 10 個 commit (了解最近的變更方向)
3. 讀取相關模組的測試 (了解期望行為)
4. 搜尋相似功能的實作 (了解設計模式)
5. 確認理解後，才開始執行任務

目標：在開始前就充分理解 codebase 的脈絡。

N-效能優化 7 個

N-01 前端效能優化

優化這個頁面的載入效能 (目前 Lighthouse 分數：[分數])：

目標：LCP < 2.5s, FID < 100ms, CLS < 0.1

請分析並修復：

- 未使用的 JavaScript (Tree shaking)
- 圖片優化 (WebP 格式、懶載入、正確尺寸)
- 關鍵 CSS 內聯 (消除 render-blocking)
- 字型載入優化
- CDN 快取設置

N-02 資料庫效能優化

資料庫效能診斷：

問題：[描述，如「每天定期查詢超過 30 秒」]

資料量：[筆數]

請分析：

- 最慢的查詢 (pg_stat_statements / slow query log)
- 遺失的索引
- 統計資訊是否過時 (ANALYZE)
- 是否有資料表膨脹 (VACUUM)
- 連線池設定是否合理

N-03 API 效能優化

這個 API 端點的 P95 延遲是 [X] ms，目標是 [Y] ms：

[貼上端點程式碼]

請找出瓶頸：

- 有幾次資料庫查詢？能合併嗎？
- 有哪些計算可以快取？
- 有哪些外部 API 呼叫可以並行化？
- 回應資料是否有不必要的欄位？

N-04 Bundle 大小優化

這個 Next.js/React 應用的 bundle 太大 ([大小] MB)，影響首屏載入：

請分析 (用 webpack-bundle-analyzer 的輸出)：

[貼上分析結果或描述]

優化策略：

- 移除未使用的套件
- 按路由分割 (動態 import)
- 用輕量替代品換掉大套件
- Tree shaking 是否正確設置

N-05 快取命中率優化

目前 Redis 快取命中率只有 [X]%，目標是 [Y]%：
快取配置：[描述目前的 TTL 和快取策略]
請分析：

- 哪些查詢最頻繁但快取命中率低？
- 快取鍵設計是否合理（太細粒度？太粗粒度？）
- TTL 設定是否合適？
- 是否有快取穿透問題（查不存在的 key）？

N-06 並行處理優化

這個批次任務處理 [N] 筆資料需要 [X] 分鐘：
[貼上程式碼]

請重構為並行處理：

- 找出可以並行的步驟
- 設計適當的並行度（不要超過 API rate limit）
- 加入進度追蹤
- 處理部分失敗（不讓一筆失敗停掉全部）

目標處理時間：[Y] 分鐘

N-07 效能監控儀表板

為 [應用程式] 建立效能監控儀表板：

關鍵指標：

- API P50/P95/P99 延遲（按端點分類）
- 每秒請求數（RPS）
- 錯誤率（按錯誤碼分類）
- 資料庫查詢時間
- 快取命中率

工具：[Grafana / Datadog / 自建]

告警：關鍵指標超過閾值時通知 [管道]。